HOW A COMPUTER WAKES UP

Operating-system development explained with pictures
for people who DON'T code



Based on “The Little Book About OS Development”
by Erik Helin & Adam Renberg (2015)

Zero coding knowledge needed - just curiosity!

? What's inside this mini-book

[1] **What even IS an operating system?** — *the boss inside your computer*
[2] **How a computer wakes up** — *power button - BIOS - GRUB - your OS*
[3] **The secret handshake** — *hello, 0xCAFEBABE!*
[4] **Talking to the screen** — *a giant board of light-up tiles*
[5] **Keeping a diary** — *the serial port logbook*
[6] **Dividing up the big room** — *segmentation*
[7] **Ding-dong! Interrupts** — *how the keyboard gets attention*
[8] **Two uniforms** — *boss mode vs visitor mode*
[9] **Fake addresses** — *paging & virtual memory*
[10] **The landlord** — *handing out memory rooms*
[11] **Unpacking a program** — *loading and running code*
[12] **Filing cabinets** — *file systems*
[13] **Ringing the service bell** — *system calls*
[14] **Doing everything at once** — *multitasking*
[15] **You made it!** — *the whole journey in one breath*

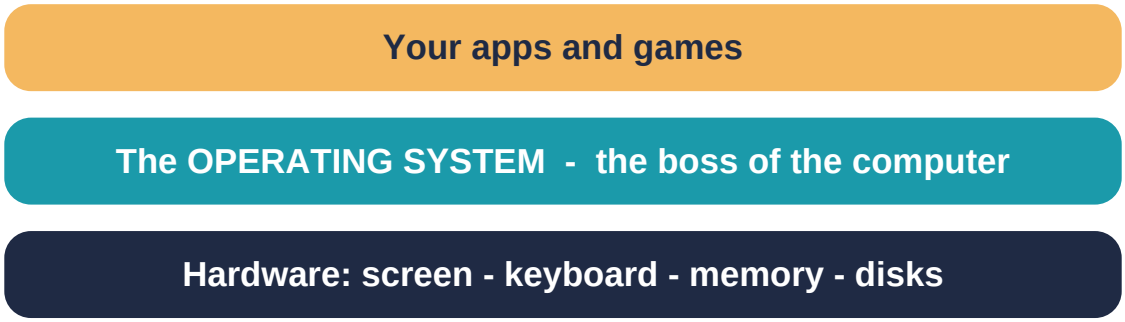
How to read this

Each chapter is short. The pictures carry the ideas, and the words use everyday language - relay races, landlords, doorbells and filing cabinets. Whenever the real book uses a scary word, we translate it here.

1 What even IS an operating system?

Imagine a brand-new computer straight from the factory. It has a brain (the processor), a big desk for papers (memory), a screen, a keyboard... but it just sits there. It has no idea how to show you anything or listen to you.

An **operating system** (OS) is the **boss program** that moves in and runs the whole place. It decides who gets to use the desk space, it draws letters on the screen, it listens to the keyboard, it keeps your files tidy, and it makes sure programs don't fight.



The book builds that middle layer from scratch - the boss itself!

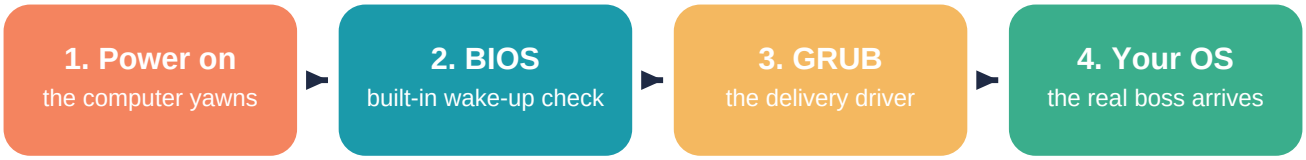
In plain words

The processor is the worker. Memory is the desk. The screen and keyboard are the doors and windows. The operating system is the manager who knows how everything fits together.

The little book you read teaches you to BUILD that boss, starting from absolutely nothing. Not by memorizing theory - by actually making it, one tiny step at a time. Its best tip: after every small step, test it. Then, if something breaks, you know exactly which step broke it.

2 How a computer wakes up

Pressing the power button does not magically start your OS. It is more like a relay race - a chain of tiny programs, each one waking up the next, bigger one:



Each one hands the baton to the next, until the operating system takes over.

Just like a relay race at school sports day!

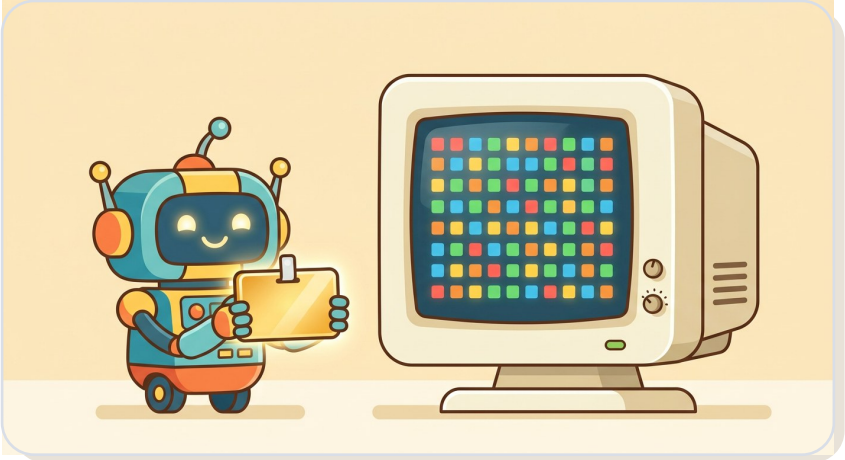
BIOS is the first runner. It is built into the hardware itself, like a wake-up alarm glued inside the machine. It stretches a little (“is the keyboard there? the screen? OK!”) and then hands over.

The **bootloader** is the second runner. Its whole life purpose is to find an operating system and get it onto the stage. Writing one is fiddly work, so the book uses a famous existing one: **GRUB**. Think of GRUB as a delivery driver who already knows where the boss's office is - he carries your OS into memory (the big desk) and says: “Go!”

Fun fact

The very first runner must squeeze into a tiny 512-byte mailbox on the disk - about the size of one short text message! That is why nobody writes bootloaders just for fun.

3 The secret handshake: Hello, 0xCAFEBABE!



Before GRUB starts your code, it checks the secret password at the door.

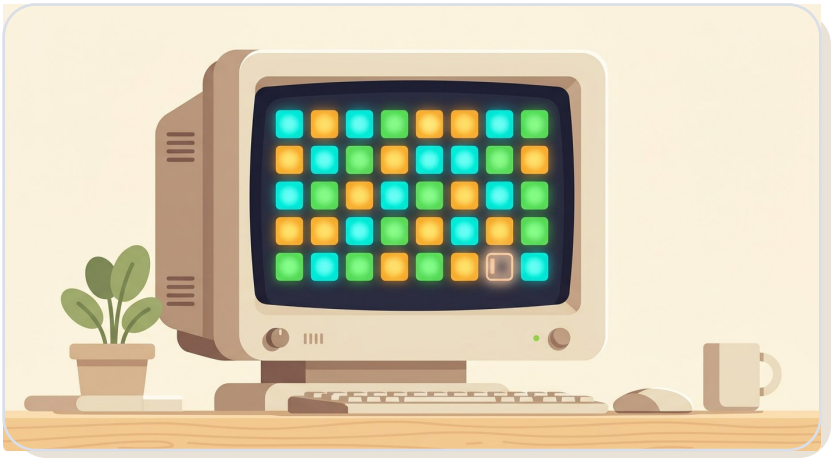
GRUB is careful. Before it jumps to your code, it looks for a **magic number** - a secret password written at the very top of your program. The password is **0x1BADB002** (hex-speak for a memorable handshake), and the tradition is to answer with a second funny number when you succeed: **0xCAFEBABE**. Yes, really - it reads like “cafe babe”. Programmers have a sense of humor too.

The tiniest possible OS does just one thing: it writes 0xCAFEBABE into the CPU's hand and waves forever. The CPU's “hands” are called **registers** - tiny name-tag slots where it holds the number it is working with right now.

How do we know it worked?

The pretend computer (a simulator called Bochs) can print out all its registers when you stop it. If you see CAFEBABE sitting in one of them - your code ran! That is the “Hello, world!” of operating systems. Smallest OS ever: it does absolutely nothing, and everyone is thrilled.

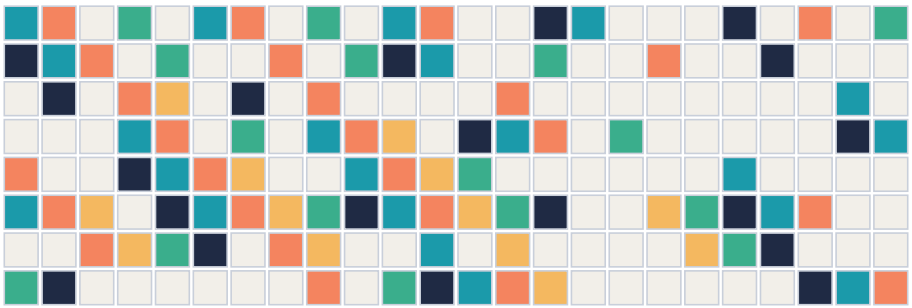
4 Talking to the screen



The text screen works like a board of light-up tiles.

How does a letter appear on the screen? The text screen is really just a **giant light board**: 80 tiles across, 25 tiles down - 2,000 tiles in total. Each tile holds one letter plus its colors, packed into 2 tiny bytes (a byte = 8 tiny on/off switches).

The screen = a light board of 80 x 25 tiles (here: a tiny sample)



Each tile holds: one letter + its colors, packed into 2 tiny bytes

Write to memory spot 0xB8000 - and the tiles light up!

Here is the sneaky trick: the screen is wired to a fixed spot in memory - spot number **0xB8000** (that is just its locker number). Write a letter's code there... and it instantly lights up on the real screen! Writing to memory IS talking to the screen.

The blinking cursor is shyer - it is not driven by memory but through a side door called an **I/O port** (two little numbered chutes, 0x3D4 and 0x3D5). Your code slips a note through the chute: “cursor, please move to tile 80”.

What is a “driver”?

Instead of every part of your OS whispering hardware numbers, you write one helpful middleman - a DRIVER. The rest of the system just says “print this sentence, please” and the driver handles the nerdy details. Same idea as a TV remote: one friendly button, complicated magic inside.

5 Keeping a diary: the serial port

When things go wrong, it really helps to know what your OS was doing. Enter the **serial port** - an old-fashioned plug that speaks in a steady stream of bytes, like a telegraph line or a walkie-talkie channel.

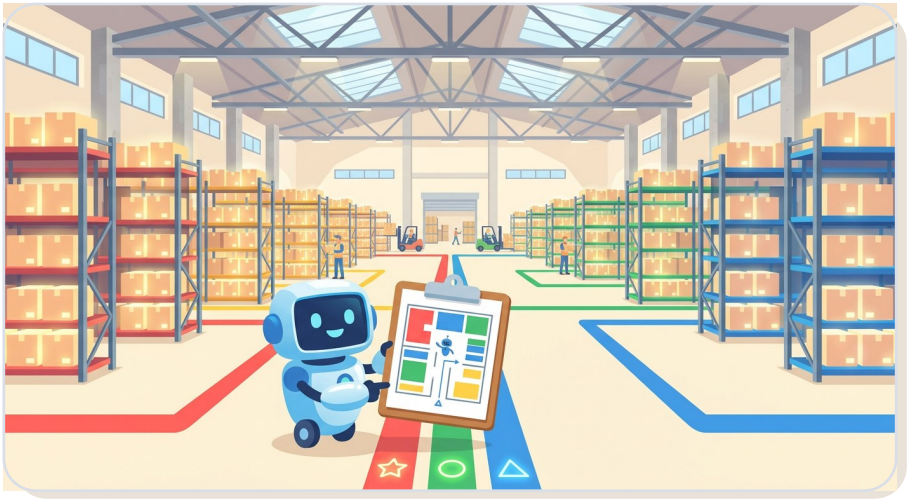
Neat trick: the pretend computer Bochs can be told to “write down everything said on that line into a file”. Boom - a **logbook**! Or like an airplane's black box: whatever your OS did, it leaves a trail you can read afterwards.

Before chatting, agree on how

Two radios must tune to the same channel before they can talk. The serial port needs setup too: how FAST we speak (the speed), how we double-check for mistakes (parity & stop bits), and how big the in-tray is (buffers). The book walks through each dial - then builds a driver with a nice DEBUG / INFO / ERROR log function, just like real software.

With a screen driver AND a diary driver, the OS can finally say where it is and what it sees. From here on, whenever we hit a bug, the diary will save us hours of guessing.

6 Dividing up the big room: segmentation



Memory: one giant warehouse - segmentation paints it into zones.

Memory is one gigantic warehouse of tiny lockers, all numbered. Without rules, anything could scribble anywhere - one grumpy program could erase everyone else's work.

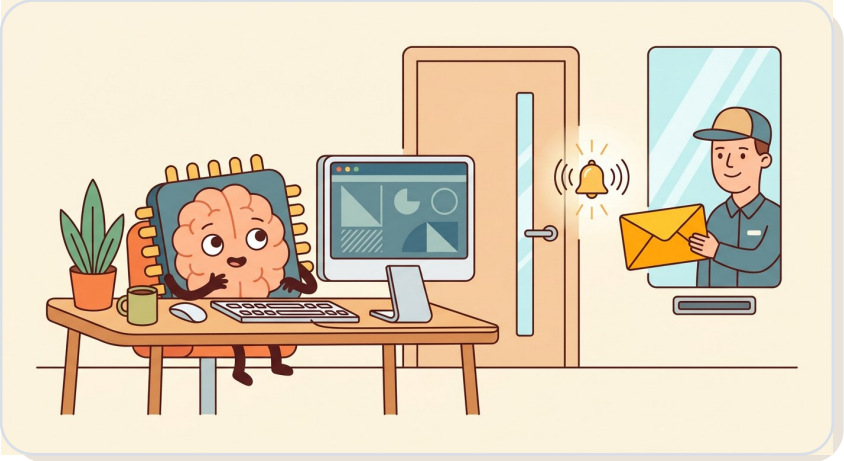
Segmentation is like painting the warehouse floor into colored zones: “program code lives here”, “data lives there”, “boss-only area”. Each zone has a starting point, a size limit, and a permission level. The CPU carries six little **name tags** (called segment registers) that say which zone it is currently allowed into.

Boss level 0, visitor level 3

x86 chips have four “privilege levels” numbered 0 to 3, where 0 is the supreme boss and 3 is the visitor badge. The OS builds its zones at level 0. Later, programs run at level 3 and simply CANNOT barge into boss areas - the hardware itself blocks the door.

Honestly? Modern systems barely use segmentation - everyone switched to the next trick (paging, coming soon). But you still need a minimal setup with the right name tags, so we do exactly that: two zones (code + data), just enough to get the privilege levels working.

7 Ding-dong! Interrupts & the keyboard



An interrupt is a doorbell: “Stop a second - the keyboard needs you!”

The CPU is a busy worker that could compute forever. But you just pressed a key! How does anyone get its attention? With a **doorbell**. In computer speak: an **interrupt**.

When a key is pressed (or a timer rings, or a wire has news), the hardware goes DING. The CPU pauses, looks up “who handles this doorbell?” in a big phone book (the **IDT** - Interrupt Descriptor Table), runs that helper routine, then returns to its own work *exactly* where it left off - mid-thought, registers and all.

The mailroom (PIC) and the keyboard's secret code

A little sorter chip called the **PIC** decides whose ring gets through (you don't want doorbells from broken devices you cannot answer). And here is a surprise: the keyboard does NOT send letters - it sends **scan codes**, like Morse tap-patterns. Your OS must translate tap #30 into “a”. Also: always say “thanks, got it!” back to the PIC, or it will not ring again!

8 Two uniforms: boss mode vs visitor mode



The castle keeps the master keys inside - visitors stay in the village.

Remember the privilege levels? Now they matter. **Kernel mode** is the manager's uniform with the master key - the OS may touch anything. **User mode** is the visitor badge - apps get a fenced play area and nothing else.

Why bother? Because apps will crash, freeze and misbehave - and when a visitor trips, the whole building must NOT fall over. In user mode, a broken app just... quietly fails, while the boss stays safe and sound.

The road ahead

To run a program in visitor mode, the book first lets one run in BOSS mode (easiest), then builds the full gate: special zones (chapter 6), fake addresses (chapter 9), and a proper switch through the CPU's “return from interruption” door - the only way to put on the visitor uniform.

9 Fake addresses: paging & virtual memory



Every tenant thinks the whole building is theirs - the manager knows better.

Here is the big magic trick of modern computers. Every program is told: “The whole address space is yours - lockers 0 to 4 billion!” Obviously there is not that much real memory. It is a polite lie.

The trick is called **paging**. Memory is chopped into equal **pages** - boxes of 4,096 bytes each. A big **catalog** (the page table) translates every fake address the program uses into the real box where the data actually lives. A tiny hardware clerk inside the CPU (the MMU) does this translation billions of times per second.

Every program thinks it owns the whole building - the catalog quietly redirects.



Knock on a locker that was never rented? BEEP! That is a "page fault".

Why fake addresses are wonderful

1) Each app gets its own privacy fence - it cannot peek at others. 2) Memory can be scattered anywhere, yet the app still sees a nice tidy hallway. 3) The boss can mark pages “read-only” or “employees only”. 4) Apps can think they own more memory than the machine even HAS.

10 The landlord: handing out memory rooms

With fake addresses flowing, somebody must track the REAL boxes: which are free, which are taken, and which hold the elevator machinery and must never be rented out. That somebody is the **page frame allocator** - the landlord of memory.

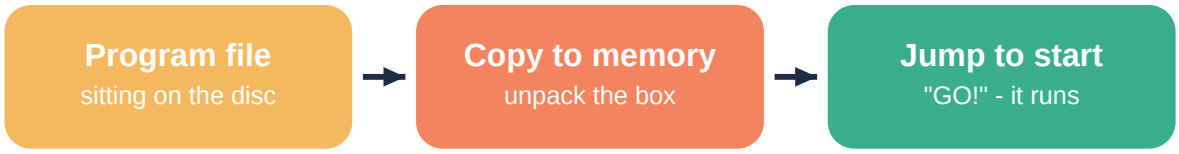
Good news: GRUB already snooped the machine at startup and hands your OS a map of the building - which floors exist, which are RAM, which are special equipment. The book's advice: use a simple **checklist** (one checkbox per box - a “bitmap”) and remember to paint the kernel's own rooms as TAKEN. (GRUB does NOT do that for you. Oops.)

And the supply closet: a heap

With a landlord in place, the OS can have its own “supply closet” (a kernel HEAP): whenever it needs a desk, a list or a name tag, it asks the closet (malloc / free in the book). The closet asks the landlord for more rooms whenever it runs out.

One clever plumbing trick from the book: what if the OS needs a new catalog page to map a box, but mapping THAT needs a catalog page...? The fix: keep one “emergency window” slot permanently free, used only to peek at a fresh box long enough to file its paperwork. Every OS has a trick like this up its sleeve.

11 Unpacking a program



The book uses GRUB "modules": the delivery driver drops the package in memory before the OS even starts. Easy peasy!

How does an app get onto the stage? A real OS reads programs off disks using drivers and file systems - lots of work. The book takes a shortcut: GRUB can already carry “packages” (called **modules**) into memory before the OS starts. The OS just finds the package address and hops to it.

The first test program is hilariously small: it writes **0xDEADBEEF** into a register and loops forever (another classic programmer joke). Run it, stop the simulator, check the log: BEEF found? Then a real, separate program ran inside your OS!

Real apps, later

Later, real apps are written in C but packed into a “flat” shape starting with a tiny 3-step “welcome mat” at the very front: call main, get the result, loop forever. A tiny linker script orders the pieces (the full how-to is in the book's chapter 11).

12 Filing cabinets: file systems



A file system is the cabinet, the labels, and the librarian rules.

Why do computers need file systems? Ask: which program starts first? Where do settings live? How does an app find its data? In the Unix world, the answer to nearly everything is: “it is a file”. The file system is the **filing cabinet** that makes that possible.

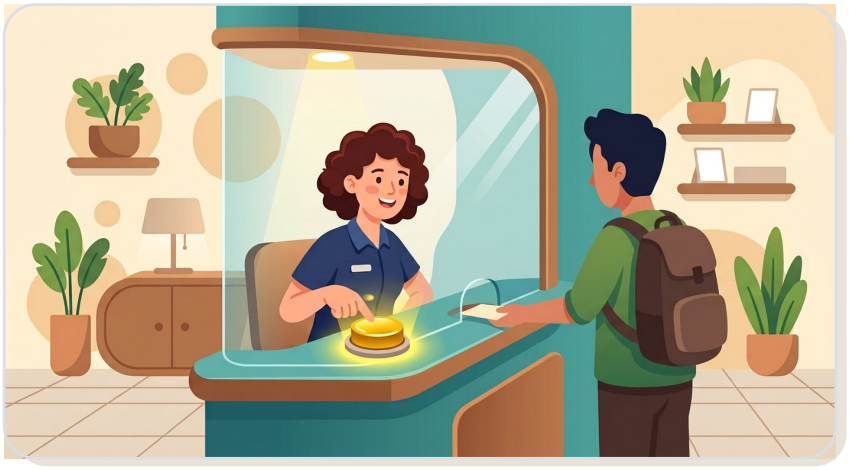
Level 1 (simple): one read-only envelope in memory. A little build-time tool stamps each file with a header (name, size, type) and packs them in a row. The kernel reads them like a roll of tickets.

Level 2 (real): when you need to WRITE files, you meet the **inode** - a library card for each file saying where its pages live. This is how Linux and friends work under the hood.

The librarian: a Virtual File System (VFS)

Should the keyboard and screen be files too? A VFS is one friendly librarian that presents EVERYTHING as folders and files, then quietly hands each “open, read, write” to the right drawer. That is why, on Linux, even your speakers live under /dev!

13 Ringing the service bell: system calls



Apps never walk into the manager's office - they press the service bell.

Visitor-badge apps cannot touch hardware or memory directly. So how do they save a file or print to the screen? They press a **service bell**: a **system call**. It is the one official way for an app to ask the boss for something.

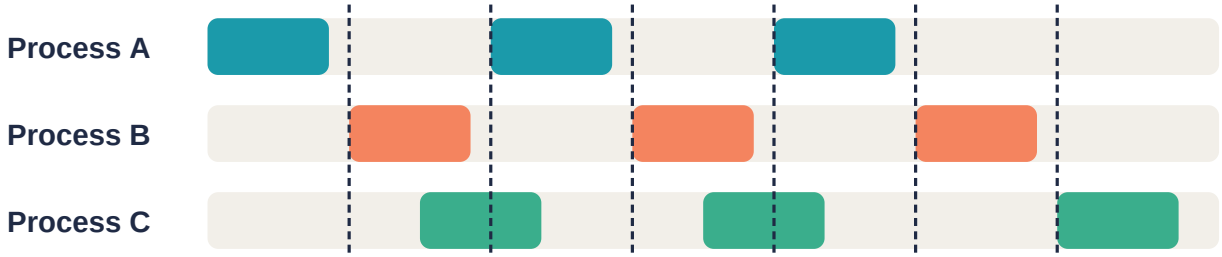
Under the hood, the app fills out a tiny request form (a few numbers saying what it wants), then rings a special doorbell built with a software interrupt. Linux's famous doorbell is number 0x80. The CPU instantly switches into boss uniform, does the job, and hands back the answer - all faster than you can blink.

The reception desk (TSS)

When a visitor rings, the CPU must know WHICH desk to switch to. It consults a tiny structure called the TSS - the reception desk address book - that says: “the boss's desk is over there”. The book sets this up right before entering user mode. Also: the doorbell must be marked “visitors may ring”, or the CPU will refuse the ring!

14 Doing everything at once: multitasking

Your music plays, your download runs, your text editor waits - all “at the same time”. But the CPU juggles only ONE thing at any instant. The trick is switching so fast that it LOOKS simultaneous - like a chef flipping between three pans.



The little clock rings - and the next one gets a turn

So fast that it LOOKS like they all work at the same time

There are two ways to switch. **Polite way (cooperative):** each app, when it can spare a moment, says “your turn!” (a yield system call). Simple and easy to debug - but one rude app can hog everything. **Fair way (preemptive):** a tiny oven timer (the PIT) rings every ~20 milliseconds and the boss FORCES a switch. Everyone gets a fair slice of time.

Switching is like pausing a video game: **every single detail** must be saved - all the registers, the exact instruction spot, the whole uniform state - so the next turn resumes mid-thought as if nothing happened. Each process also gets its OWN notepad (its own stack), so nobody scribbles on anyone else's work.

Pro tip from the book

Build the polite version FIRST. Because it is deterministic (always the same steps in the same order), bugs are far easier to catch. Only then add the oven timer. Slow and steady wins the race - the authors built their whole OS in six weeks exactly this way!

15 You made it! The whole journey

That is the entire book - from a dead piece of metal to a little world that can run programs, show text, listen to the keyboard and juggle tasks. Let us re-tell the whole story in one breath:

The journey in one breath

A computer wakes up (BIOS), hands over to a delivery driver (GRUB), who starts YOUR boss program after checking the secret handshake (0xCAFEBABE). The boss paints the light-board screen, opens a diary line, draws zones and permissions on the warehouse floor (segmentation), installs doorbells (interrupts), invents fake addresses (paging), rents out memory rooms (the allocator), unpacks programs, files paperwork (file systems), opens a service counter (system calls) and learns to juggle (multitasking).

The real book says something wise: build in **tiny steps**, and after every step, TEST it. That habit - more than any single line of code - is how six weeks of steady work turns an empty machine into a working operating system.

And if you ever get stuck? The authors point to the friendly OSDev wiki community, the book's own issue tracker on GitHub, and - above all - patience. Every expert was once exactly where you are now.

The end... and the beginning

Now you know what every computer does the instant you press its power button - and roughly how people build one from scratch. If reading this made you curious about coding, that is the perfect sign. The next tiny step is yours to pick!